JOURNAL – PRACTICAL 8

Context-Aware System using JCAF (Java Context-Awareness Framework)

1. Aim

To design and develop a simple context-aware system using the Java Context-Awareness Framework (JCAF) that creates a Person entity, registers it with the JCAF Context Service, adds a location context, and detects a context change.

2. Learning Outcomes

- Understand the basic idea of context awareness.
- Set up the JCAF 1.5 deployment package.
- Start the JCAF Context Service from the Windows Command Prompt.
- Create and run a Java project in Eclipse.
- Create a custom Person entity by extending GenericEntity.
- Add a Location context using the Located relationship.
- Observe the context-change callback in the Eclipse Console.

3. Software / Requirements

Requirement	Recommended / Used
Operating System	Windows
Java	JDK 17 (javac/java 17.x)
IDE	Eclipse IDE
Framework	JCAF 1.5
JCAF package	jcaf.v1.5.deploy_j2se.zip (deployment package)
Optional package	jcaf.v1.5.examples.zip (examples only)

4. What the Practical Does

The program models a person named Harshita as a context-aware entity. The person is registered with JCAF. A location named Computer_Lab is then attached to that person through a Located relationship. When the context service reports the change, the Person class receives a ContextEvent and prints the context-change information.

5. Step-by-Step Setup and Execution

Step 1 – Check Java Installation

Open Command Prompt and type:

```
java --version
javac --version
```

Both commands should show Java 17 (or a compatible JDK). The screenshots supplied for this practical show java 17.0.14 and javac 17.0.14.

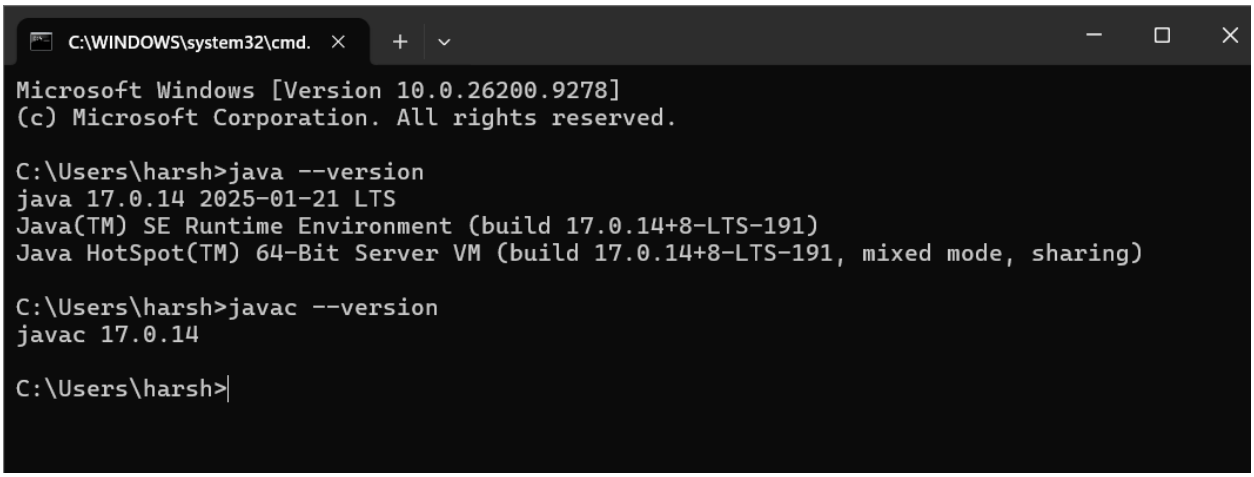


Figure 1 – Checking Java and javac versions.

Step 2 – Download JCAF

Open the JCAF project download page on SourceForge and download the JCAF 1.5 J2SE deployment package. The required file is the deployment ZIP, shown in the supplied screenshot as "jcaf.v1.5.deploy_j2se.zip".

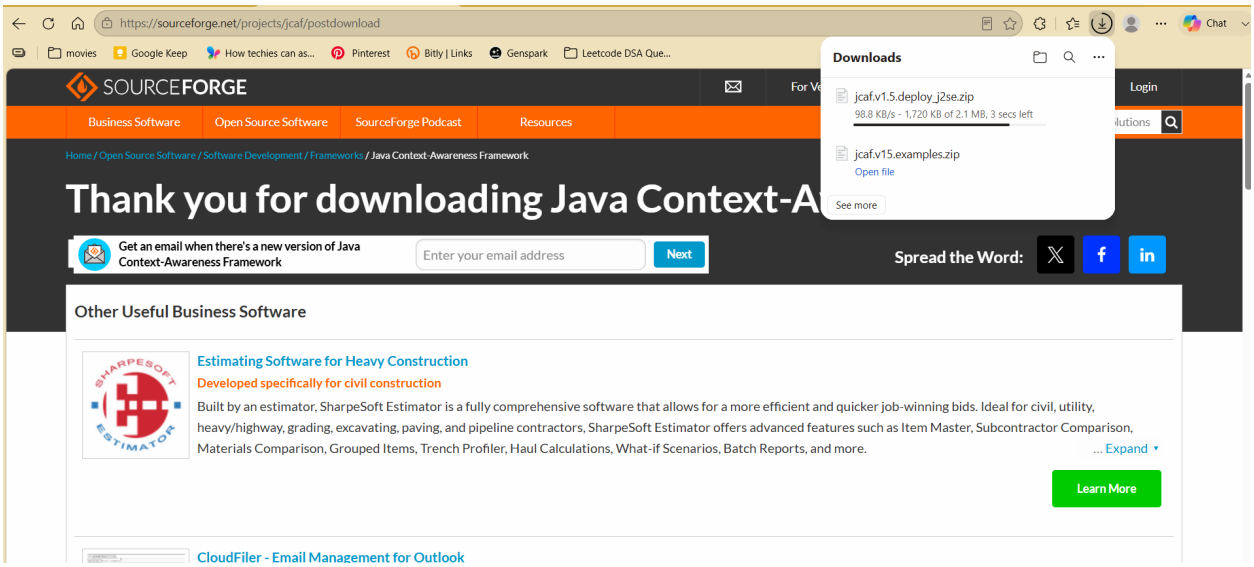


Figure 2 – JCAF deployment package being downloaded.

Step 3 – Extract the JCAF ZIP File

After downloading, open the Downloads folder and extract jcaf.v1.5.deploy_j2se.zip. The extracted folder should contain a deploy_j2se folder.

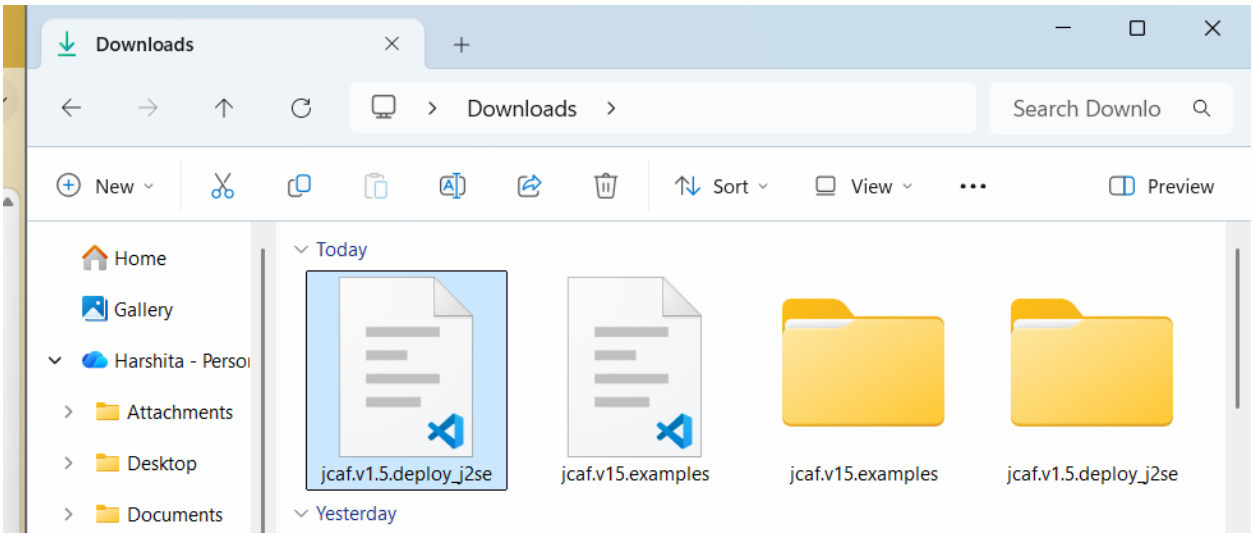


Figure 3 – Downloaded JCAF files and extracted folders.

Step 4 – Locate the JCAF Deployment Folder

Open Command Prompt. The easiest method is to copy the complete path of the deploy_j2se folder and use the cd /d command. This is important because the JCAF batch file and JAR files are stored in this folder.

```
cd /d "C:\Users\<username>\Downloads\jcaf.v1.5.deploy_j2se\deploy_j2se"
dir
```

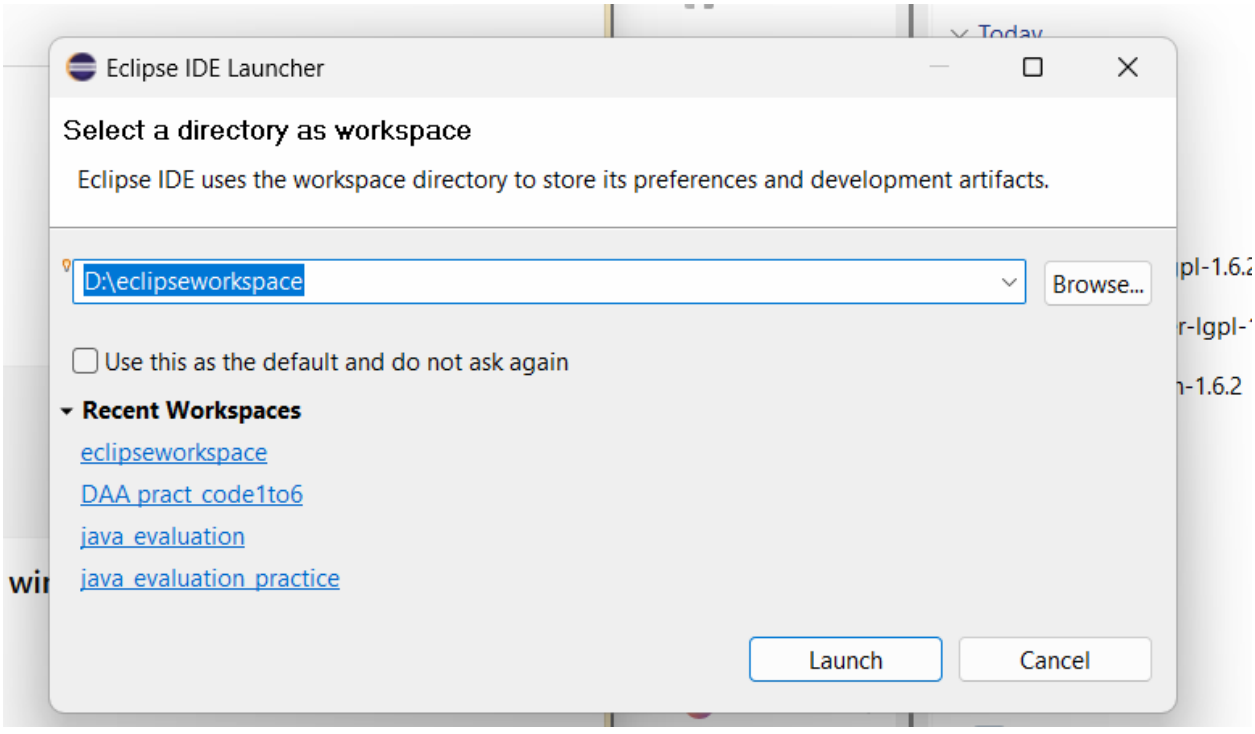


Figure 4 – Navigating to the JCAF deploy_j2se folder and checking its contents.

Students should see files similar to:

- ContextService.bat
- jcaf.v15.jar

- gson-1.5.jar
- jackson-core-lgpl-1.6.2.jar
- jackson-mapper-lgpl-1.6.2.jar
- jackson-mrbean-1.6.2.jar
- jcaf.examples.jar
- java.policy
- jettison-1.0.1.jar
- org.json.jar
- org.restlet.jar
- runRegistry.bat

Step 5 – Start the JCAF Context Service

Keep Command Prompt open. From inside the deploy_j2se folder, execute the following command:

```
ContextService.bat practical8
```

Here, practical8 is the service/registry name used by the Java program. Do NOT close this Command Prompt while running the Eclipse program.

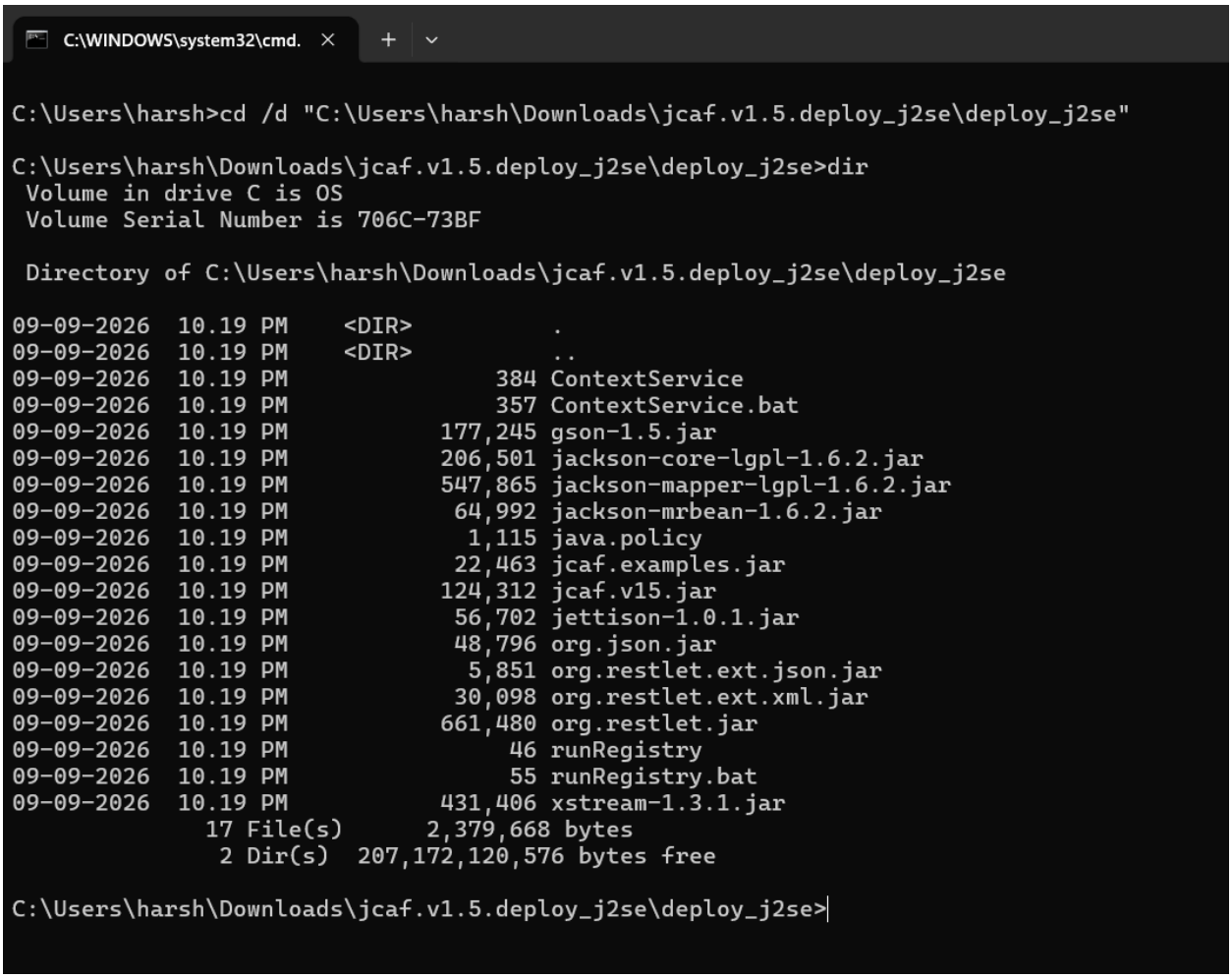


Figure 5 – Starting JCAF Context Service successfully.

Step 6 – Understand the Warning Messages

With JDK 17, JCAF 1.5 may display warnings such as 'A terminally deprecated method in java.lang.System has been called' and warnings related to System.setSecurityManager. These messages come from the old JCAF library, not from the student's Person or ContextAwareSystem code.

For this practical, the important confirmation is that the service reaches a successful state such as 'Context Service started and registered' and/or 'Ready!'. If the service successfully binds to the RMI registry, the warning itself does not mean that the practical has failed.

If an AccessControlException appears before successful registration, first make sure the JCAF service is being started from its own deploy_j2se folder and that the supplied JCAF java.policy file is present. The supplied successful run shows that the service can register correctly.

Step 7 – Open Eclipse and Create the Project

1. Open Eclipse IDE.
2. Select or create a workspace, for example D:\eclipseworkspace.
3. Create a Java Project named JCAF_Practical8.
4. Use the installed JDK 17.
5. Inside src, create a package named practical8.

```
C:\Users\harsh\Downloads\jcaf.v1.5.deploy_j2se\deploy_j2se>ContextService.bat practical8
The Java Context-Awareness Framework - JCAF
  version : 1.5
    (c)   : Jakob E. Bardram
    home  : http://www.daimi.au.dk/~bardram/jcaf/
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by dk.pervasive.jcaf.impl.ContextServiceImpl (file:/C:/Users/harsh/Downloads/jcaf.v1.5.deploy_j2se/deplo
y_j2se/jcaf.v15.jar)
WARNING: Please consider reporting this to the maintainers of dk.pervasive.jcaf.impl.ContextServiceImpl
WARNING: System::setSecurityManager will be removed in a future release

Restarting Context Service........
  Binding servers in RMI Registry...
    -> rmi://localhost/practical8  ok!
  reg. name : rmi://localhost/practical8
   hostname : 192.168.29.218
        URI : rmi://192.168.29.218/practical8

Creating and registring SCAP Gateway (Simple Context-Awareness Protocol)
SAP Gateway is running ...
  version : 2.0
  port    : 15338
Context Service started and registered.

Ready!


Attaching and starting web server for RESTful communication
  version : 1.1
  author  : Karoly Szanto
  email   : ksza@itu.dk

  using condification strategy: [ XML with XStream ]
Sep 09, 2026 10:40:49 PM org.restlet.engine.http.connector.HttpServerHelper start
INFO: Starting the internal HTTP server on port 8185
```

Figure 6 – Eclipse project/package structure used for the practical.

Step 8 – Add JCAF Libraries to the Eclipse Project

The Java program imports classes from dk.pervasive.jcaf. Therefore, the JCAF JAR files must be added to the project build path.

6. Right-click JCAF_Practical8 → Build Path → Configure Build Path.
7. Open Libraries → Classpath.
8. Choose Add External JARs.
9. Browse to the extracted deploy_j2se folder.
10. Add jcaf.v15.jar and the dependency JAR files supplied in that folder (JSON, Jackson, Restlet, Gson, etc.).
11. Click Apply and Close.

If the imports dk.pervasive.jcaf.* are already resolved in Eclipse, this step has already been completed and does not need to be repeated.

Step 9 – Create Person.java

In src → practical8, create a Java class named Person.java and use:

```
package practical8;

import dk.pervasive.jcaf.ContextEvent;
import dk.pervasive.jcaf.entity.GenericEntity;

public class Person extends GenericEntity {

    private String name;

    public Person(String id, String name) {
        super(id);
        this.name = name;
    }

    public String getName() {
        return name;
    }

    @Override
    public void contextChanged(ContextEvent event) {
        System.out.println("================================");
        System.out.println("CONTEXT CHANGE DETECTED");
        System.out.println("Person: " + name);
        System.out.println("Entity ID: " + getId());
        System.out.println("Context item: " + event.getItem());
        System.out.println("================================");
    }
}
```

Step 10 – Create ContextAwareSystem.java

Create another Java class named ContextAwareSystem.java in the same practical8 package:

```
package practical8;

import dk.pervasive.jcaf.ContextService;
import dk.pervasive.jcaf.impl.ContextServiceImpl;
import dk.pervasive.jcaf.item.Location;
import dk.pervasive.jcaf.relationship.Located;

public class ContextAwareSystem {
```

```
public static void main(String[] args) {

    try {
        // Create a person entity
        Person person = new Person("person1", "Harshita");

        System.out.println("Person created.");
        System.out.println("Name: " + person.getName());
        System.out.println("ID: " + person.getId());

        // Get the JCAF Context Service
        ContextService contextService =
                new ContextServiceImpl("practical8");

        // Register the person with JCAF
        contextService.addEntity(person);

        System.out.println();
        System.out.println("Person registered with JCAF.");
        System.out.println();

        // Create a location
        Location location =
                new Location("Computer_Lab");

        // Create a Located relationship
        Located located =
                new Located("Practical_8");

        // Add location context to the person
        contextService.addContextItem(
                person.getId(),
                located,
                location
        );

        System.out.println();
        System.out.println("Location context added.");
        System.out.println("Location: Computer_Lab");

    } catch (Exception e) {
        e.printStackTrace();
    }
```

```
    }
}
```

Step 11 – Run the Correct File

Run ContextAwareSystem.java, NOT Person.java. Person.java is an entity/callback class and does not contain the main() method. ContextAwareSystem.java contains public static void main(String[] args), so it is the entry point.

In Eclipse:

12. Open ContextAwareSystem.java.
13. Right-click inside the editor.
14. Select Run As → Java Application.
15. Keep the JCAF Context Service Command Prompt running in the background.

Step 12 – Expected Output

The Eclipse Console should show output similar to:

```
Person created.
Name: Harshita
ID: person1

Person registered with JCAF.

Location context added.
Location: Computer_Lab

================================
CONTEXT CHANGE DETECTED
Person: Harshita
Entity ID: person1
Context item: null
================================
```

6. Explanation of the Program

Person entity: Person extends GenericEntity, so JCAF can treat a person as a context-aware entity.

Person ID and name: person1 uniquely identifies the entity, while Harshita is the human-readable name.

ContextServiceImpl: Connects the Java program to the JCAF context service using the service name practical8.

addEntity(person): Registers the Person entity with JCAF.

Location: Represents the context value Computer_Lab.

Located: Represents the relationship used to associate the person with a location.

addContextItem(...): Adds the location context to the registered person.

contextChanged(...): Callback in Person.java that executes when JCAF reports a context change.

7. Important Student Instructions

- Start the JCAF Context Service FIRST from Command Prompt.
- Do not close the ContextService.bat window while executing the Eclipse program.
- Use the exact service name practical8 in both ContextService.bat practical8 and new ContextServiceImpl("practical8").
- Run ContextAwareSystem.java because it contains main().
- Person.java is supporting code; it should not be run as a Java Application.
- If Eclipse shows unresolved JCAF imports, check the JARs in Build Path.
- Warnings about SecurityManager on JDK 17 are generated by the old JCAF framework. Focus on whether the service successfully registers and the program produces the expected output.

8. Troubleshooting

Problem	Solution
'ContextServiceImpl(String)' constructor error	Use new ContextServiceImpl("practical8") instead of new ContextServiceImpl().
JCAF package/import errors	Add jcaf.v15.jar and the required dependency JARs to Eclipse Build Path.
AccessControlException / 127.0.0.1:1099	Restart the JCAF service from the deploy_j2se folder. Ensure java.policy is present and use ContextService.bat practical8.
Service not found / connection failure	Make sure the Command Prompt running ContextService.bat is still open and the service name is practical8.
No context-change output	Run ContextAwareSystem.java and verify that addContextItem() executes after addEntity(person).
Only warnings appear	SecurityManager deprecation warnings are expected with older JCAF on modern Java. Verify that the service reaches 'Ready!' / 'Context Service started and registered.'

9. Conclusion

Thus, a basic context-aware system was successfully implemented using JCAF. A Person entity was created and registered with the JCAF Context Service, a Computer_Lab location context was associated with the person, and the context-change event was detected and displayed in the console.

10. Viva Questions

1. Motive of the Practical — Why Are We Doing This?

The main purpose of this practical is to understand **how a computer system can know about the situation around an object/person and react to it**.

In simple words, we are teaching a Java program:

“This is a person, this person is registered in the system, and right now this person is in the Computer Lab. If the person's context changes, tell me.”